

Project 3 Final Project

May 4th, 2025

Ethan Senatore

I. OVERVIEW

Throughout the entirety of the course we have been slowly gearing towards being able to autonomously navigate an environment using a quadrotor. Project 3, the final project, asked the students to combine all of their previously implemented code in order to autonomously plan and navigate the maps provided. This project worked off of the foundation of material laid out over the entirety of the semester. The varying topics include state-space discretization, search algorithms, trajectory generation methods, and state estimation. The culmination of all these concepts is formed into a well-balanced project asking the students to combine all that has been learned in the course.

A. Path Planning

For the path planning portion of the problem, I leveraged an A^* search algorithm conducted over a discretized version of the map using the L2 distance as the heuristic cost. To maximize the performance and safety of the search, I tuned the resolution of the occupancy map and the closest margin of distance allowed to an obstacle. I settled on the following values:

$$\text{Resolution} = [0.25m \quad 0.25m \quad 0.25m] \quad \text{Margin} = 0.5m$$

B. Trajectory Generation

Once a feasible path plan is found, a trajectory is generated for the quadrotor to follow. I accomplished this by using a minimum jerk-constrained trajectory approach in which multiple splines are stitched together to complete a full trajectory. This is done quite simply by enforcing continuity between splines and setting the higher derivatives to 0, enabling us to construct an $AX = B$ system, where A is the polynomial matrix and B is the constraint matrix. Solving for the missing coefficients, X , is done quite simply through LU decomposition in the *numpy* library.

These coefficients are then used to calculate the desired position, velocity, acceleration, snap, and jerk which are then fed back into the controller. I elected to try and maintain a velocity of $4m/s$.

C. State Estimation

For state estimation, an onboard pseudo-vision package uses a visio-inertial odometry setup. This is accomplished using an error state Kalman filter (ESKF) which leverages IMU and stereo matching information to best estimate the quadrotor's state. A nominal state, modeled around the IMU measurements, is maintained while leveraging the innovation from the stereo based odometry to improve its own estimate.

II. IMPLEMENTATION

Fortunately, for the final project, I made little to no changes to the `world_traj.py` code. Initially, as I had tried for project 1.3, I attempted to replicate the results from Liu et al. [LAMK17]. Unfortunately, I was not able to solve an issue regarding searching over an infinite state space. I attempted using acceleration motion primitives to search over position-velocity space. By adding the velocity dimension to the state space, the dimensionality goes from $\mathbb{R}^3$ to $\mathbb{R}^6$ which greatly increases the size of the state space. Moreover, if the search algorithm is not correctly implemented or inadmissible heuristics are used, then the search can last forever. Inevitably, I was unable to develop an adequate search solution to leverage their findings.

The only change I had made to my existing code was a minor fix regarding the last few moments before the trajectory had finished. Initially, I simply returned a geometric trajectory directing the quadrotor to the goal. However, this proved to be a source of my issues and I simply returned a flat output of the goal position with all other parameters set to 0. This allowed me to increase the attempted maintained speed of the trajectory from $3.5m/s$ to $4m/s$, which in turn proved to be quite successful.

III. RESULTS

I was pleasantly surprised with the results that I was able to achieve. As can be seen in Table I, I proved to have achieved relatively short planning and flight times for each map. I greatly improved upon my previous times from project 1.3 by beating my best total flight time combined across all six maps by 63.95 (s). I can attribute this to the minor tweaks made in the code to allow for a greater maintained velocity. When reflecting on the problem, the clear discrepancy from projects 1.3 and 3, lie completely in the state estimation. During project 1.3, we naively assumed a perfect state understanding which is completely impossible to replicate in real life. Project 3 imposed harder constraints on us by forcing us to use state estimation techniques to guess what our state may be. This discrep-

ancy in difficulty, is what further underscores my surprise to the drastic improvement in performance. By slightly decreasing the resolution and margin and tweaking the final approach to the goal a lot was accomplished.

Map	Planning Time	Flight Time
Maze	0.78 (s)	7.03 (s)
Over Under	2.07 (s)	9.56 (s)
Window	5.67 (s)	7.71 (s)
Slalom	3.18 (s)	19.72 (s)
Stairwell	4.65 (s)	9.96 (s)
Switchback	2.54 (s)	19.66 (s)

TABLE I: FINALIZED GAIN VALUES AFTER TUNING

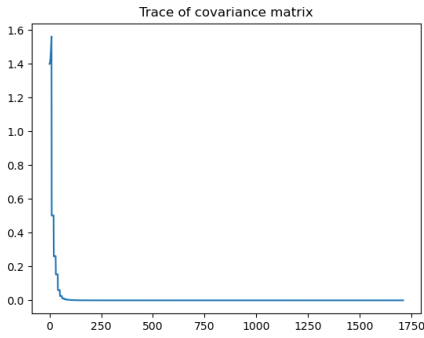


FIGURE I: TRACE OF THE STATE COVARIANCE MATRIX

These results would not have been possible if my state estimation implementation was incorrect. Figure I depicts the trace of the state covariance matrix illustrating how the quadrotor quickly converged to a high confidence estimation through the use of the ESKF. Figures II and III, depict the trajectories generated on the **Maze** map. In both figures it can be seen how a safer path was generated. However, the increased velocity enabled the quadrotor to veer elegantly through the maze. I had attempted to decrease the margin even more to try and *shoot the gap* in the *Maze* map, but by doing so cost me performance on other maps.

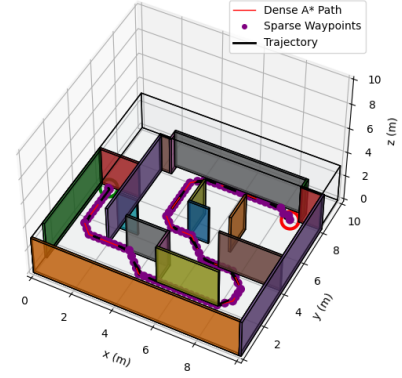


FIGURE II: A* PATH, TRAJECTORY WAYPOINTS, AND TRAJECTORY GENERATED

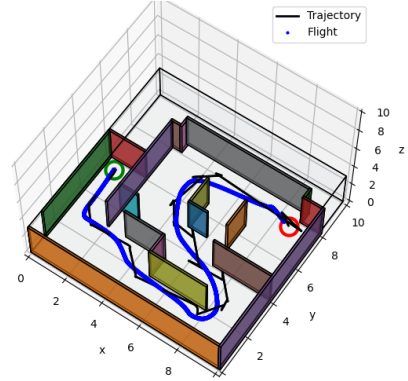


FIGURE III: TRAJECTORY AND FLIGHT PATH

REFERENCES

- [LAMK17] Sikang Liu, Nikolay Atanasov, Kartik Motta, and Vijay Kumar. Search-based motion planning for quadrotors using linear quadratic minimum time control. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 2872–2879. IEEE, 2017.